String width library Writeup ODDITY X home assignment Elad Elya
VERSION2 PART IS RIGHT AFTER V1 ON PAGE 4
V1:
Start with some research about the problem - I realized the main problem is that the UNICODE WIDTHS TABLE is huge, and the main goal is to find a better way to calculate string width than loading the whole table and saving huge hashmap (codes->width) in the memory. The tradeoff is having a fast yet small library, rather than very fast but takes lots of memory.

Also a single emoji of a family for example might be many chars with another joiner (with zero width) chars, and to analyze the width a decomposition is needed.

More I checked about the structure of a UNICODE string, and I learned that to break a string to graphemes, *VERY GENERALLY* I need to look for each char if the next char is a zero width joiner. If so, the next char is still part of the current grapheme. Else, the current grapheme has ended and now we can move on to analysing its length. Looks like there is a tool that does so - Intl.Segmenter. Making a custom one looks complicated because of many special cases (it's not really just if there is a ZWJ then continue, else end of grapheme) and redundant. It might be considered so the library can run in older versions of JS that don't have this implementation.

So far, I saw that generally Emojis are 2 units width, and non Asian letters are 1 unit width, and there is a standard named East Asian Width that shows which East Asian letter are 2 units width and which aren't, and they are ordered comfortably in a sequence of same width (All 2-width letters are in specific ranges, and not distributed randomly).

Another thing that should be taken into account is that strings might have design codes (like text color) and those codes need to be ignored (ANSI / Escape Codes). There are tools to handle that - strip-ansi for example. Making a custom one probably is not that bad, we'll see later what's more efficient.

Also there are edge cases, letters that have different widths on different terminals, need to see if possible to check the terminal type or something and know at runtime how the grapheme should be handled (after a check it looks like there is no such tool). Also need to think about options that need to be added, like if the user wants to ignore \n or such.

—----------

After understanding the problem, I can move on to solving it.
So loading the whole Unicode Widths Table is not the solution, although it's the easiest approach. It will make the library way too big.

So when we get a string, we first want to clean the ANSI part of the string, since they don't "cost" width. So we will need a strip ansi function (strip-ansi or custom).

Then, separate the string to its graphemes (Intl.Segmenter or custom).

And now the core of the code: calculating width for each grapheme.

We iterate over the grapheme components:

1. If it's a ZWJ (zero width joiner), or such, return 0.

Create a set of all these characters and check if the current char is in it at O(1).

2. If an emoji is involved, then the terminal will use 2 width units to print the whole grapheme, unless a vs15 (force 1 width) unicode is part of the grapheme.

Save list of ranges of emojis, as will be explained in the next lines. O(log(k)) where k is the ranges amount. I know there is a method that checks emojiness with regular expressions, but using the list is very efficient since the log of the number of ranges will be basically O(constant), and it's more consistent with the next part architecture. Also emojis length can vary depending on the chars that come after them in the grapheme (VS15, VS16). So when we will get to the code of this part a further explanation will be.

3. Then East Asian check, look at the East Asian Table and see whether it's a wide char or not.

Note that no need to save the whole actual table, just the ranges of unicodes that are wide, and we save them ordered in increasing order, so we can lookup at the table for a specific value in O(log(k)) iterations (where k in the amount of ranges) with simple binary search.

4. If none of these, it's a regular 1-width char and return 1 as the width.

Note that all the implementations mentioned are just the general idea and in the middle of writing the code probably many things will change :)

—----------

Implementing:
Starting with the Emojis and East Asian ranges:
The Unicode table can be changed. Thus, the library should have an engine that calculates the ranges at build time, and maybe add an option that lets the user give their own ranges if they don't want/can't rebuild. This option should be marked as not recommended if I'll add it eventually (Elad from the future say better option is that the user will manually supply the updated EastAsianWidth.txt and emoji-data.txt original unicode files).
For the first version, it's better to hardcode the ranges.
From unicode.org we can get the ranges for now:
The whole list of emojis:
https://www.unicode.org/emoji/charts/full-emoji-list.html
List of Unicodes for basic and non complex emojis:
https://www.unicode.org/Public/UCD/latest/ucd/emoji/emoji-data.txt
List of Unicodes for East Asian Width:
https://www.unicode.org/Public/UCD/latest/ucd/EastAsianWidth.txt
(I treated the ambiguous width as one for now, an option that gives the user control of that is nice to add)
Libraries that can give us the current ranges can be found here:
https://cldr.unicode.org/#TOC-What-is-CLDR-
For now I'll hardcode the ranges.
The code is pretty straightforward.
ranges.ts has the hardcoded ranges, and the binary search function that checks whether a specific unicode is within one of the ranges.
index.ts just exports out the main function, stringWidth.
width.ts has the core logic.
The code has notes that explain it well.

utils.ts just gives the stripAnsi function.

Edge/Ignored cases

1. If a string has both vs15 and vs16 unicodes, it will act as it has width of 1.
2. If an emoji is involved in a grapheme, its width is 2 (unless there is a v15 unicode) without considering emojis that might have default width 1. (☀, ♣ for example).
3. Control characters such as `\t`, `\n`, `\r`, `\0`, and bidi control characters (RTL, LTR..) currently have width 1. Their handling should be made explicitly.
4. In case more emojis are added… the ranges are hardcoded as mentioned. Marked cause we took it into account the simplicity over correctness tradeoff to supply fast something the works well for almost all cases.
5. Unsupported graphemes, for example {english_letter}+{zwj}+{emoji} might be printed as two chars with total width 3, although the function for a single grapheme will return. That shouldn't be a problem since the Intl.Segmenter should split to proper graphemes.
6. Letter + vs16 will return 2, although it has width 1
   // fixed //
7. some combining marks outside the currently hardcoded zero-width ranges (like) may return width 1 when used alone. When attached to a character, Intl.Segmenter groups them into the same grapheme.
   I just saw that "`\u0301\u0308`" has width 0, although it does take space on the screen without a character. So it's a bit inconsistent. Probably something wrong with the hardcoded ranges.
8. " 🏴 " is a flag but has width 1. The program returns width 2.
   // fixed //
9. As mentioned, returns 1 for ambiguous-width east asian chars. Not necessarily a problem but worth mentioning.
10. Unpaired surrogate code units have width 1. Usually they are printed as a question mark of width 1 so it's ok, just nice to mention as well.

(Green marked sections are fixed or not really a problem just worth mentioning)

Changes to fix section 8:
So apparently flags are two Regional Indicator Symbols connected. So just one should not count as a regular emoji with width 2.
So need to modify the EMOJI_RANGES in ranges.ts and remove `{ start: 0x1F1E6, end: 0x1F1FF }, // Regional Indicator Symbols (Flags)`
And take care of flags specifically.
Add a function that checks wether a unicode is a regional indicator:

```
export function isRegionalIndicator(codePoint: number): boolean {
    return codePoint >= 0x1F1E6 && codePoint <= 0x1F1FF;
}
```

And keep track of regionalIndicatorCount in the grapheme processing.

/fixed/
I think at that point, after I made some tests, the first version can be released 🙂
V2:
So there are still the three cases above (1-3) that are problematic or need an update, and of course note #4 that is the main goal of version 2 to solve - not hardcoding the ranges but get them at build time from unicode.org (of course not at every call cause it will ruin the runtime..). Also 7 and 9 should be addressed.
My ideas for now:
1 - just use indices and match the standard of which comes last, instead of that vs15 always "wins".
2 - will be fixed while making correct lists at stage 4.
3 - add an API option that allows to define behavior of such chars.
4 - in the next paragraph.
7 - separate between invisible unicodes to visible that are like a mark on a letter, and if there is not letter assign in width of 1.
9 - in the process of solving 4 saving range of ambiguous unicodes as well.

I am considering two approaches - and a third one that lets the user use both -
The first requires an internet connection for the build part, which is reasonable to expect from the user since the unicode table changes every once in a while and it is not really needed to rebuild frequently, just very rarely.
The second approach is that the user will provide the files from unicode
https://www.unicode.org/Public/UCD/latest/ucd/emoji/emoji-data.txt
https://www.unicode.org/Public/UCD/latest/ucd/EastAsianWidth.txt.
This is less professional, but doesn't require an internet connection.
I think since the rebuilds need to happen very rarely, I'll use just the first approach.

Just before jumping into the code I made another research just to make sure I'm not missing anything, and to conclude those files are really the only files I need to know all the 2-width chars.
I also saw that in order to know all the zero-width chars, I need to check out a different file, but I saw that the standard is to hardcode those chars, and it make sense because zero width chars aren't supposed to be added or changed like the emojis or width-2 chars do. I'll see if it is needed as I go.
So I made a small research about the files in order to know what we really need from them:
Emoji-data.txt -
Holds for a point code Emoji, if it's an emoji, and Emoji_Presentation which tells us the default presentation of it - its great to solve #2. The rest of the info there is irrelevant for the width.
EastAsianWidth.txt -
The relevant info is stored as: F, W, A, NA, H, N
f, w are wide (2-width), A is ambiguous (what we need to store to handle #9), and the last three are just 1-width size and can be ignored so they will go to the default of width 1.
So the lists we will need to store will be:

1. List of width 2 anyways. Emojis with wide default presentation, and wide east asian chars. Those will add 2 to the total width anyways.
2. List of emojis with narrow default presentation, so if there is a code from there + vs16 we can tell the width should be 2 and not 1.

3. List of ambiguous east asian chars, and handle them as the user requests.

So Gemini provided this amazing idea of creating the ranges from the files.
The idea is as follows:
First, each unicode can be in multiple sections, in the emojis file it can be in both emoji section and emoji_presentation (width 2 for us) section for example.
So since we need to know 4 keys on each unicode, emojiness, emoji_presentationess, wideness and ambiguity, instead of creating an object that hold 4 booleans for each unicode we can just present each unicode with 4 bits only - where if the lsb is on, it is an emoji, if the second lsb is on, then it's an emoji with default width 2, if second msb is on, it's wide char and if msb is on it is an ambiguous char.
So after that, we create an array of the length of the amount of all unicodes possible. So arr[x] holds the details about the unicode x.
const unicodeSpace = new Uint8Array(0x10FFFF + 1);
Since we only need 4 digits for each unicode, we defined it as Uint8Array (each cell hold value from 0-255), so the array won't take lots of useless memory.
(actually I can see an optimization here where each cell will hold two unicodes, and we can save half of the used memory. I won't do it now but it's a nice thing for future optimizations.)
All the cells start as 00000000.
When we look at the files template, we can see that they have the same structure:
From eastAsian:

```
0028            ; Na # Ps          LEFT PARENTHESIS
0029            ; Na # Pe          RIGHT PARENTHESIS
002A            ; Na # Po          ASTERISK
002B            ; Na # Sm          PLUS SIGN
```

From emojis:

```
2618          ; Extended_Pictographic# E1.0   [1] (☘)       shamrock
261D          ; Extended_Pictographic# E0.6   [1] (☝)       index pointing up
2620          ; Extended_Pictographic# E1.0   [1] (☠)       skull and crossbones
```

Sometimes a range is given rather than a single unicode value (002E..002F for example).
So we need to extract the start of the range and the end of it (if it exists).
^ start of a line
([0-9A-F]+) sequence of numbers and letters from A-F (a Hexa number) and capture it
Then try to match exactly two dots \.\. And capture another hexa number,
Then match any number of spaces, then a ';' and again any number of spaces, and then an identifier (a name with english letters and or underscores).

```
    const lineRegex =
/^([0-9A-F]+)(?:\.\.([0-9A-F]+))?\s+;\s+([A-Za-z_]+)/;
```

So for example,
The first line in the first picture will catch 0028 and the string "NA " (catches till the #).
So this way match a line with this lineRegex will return a list with the bottom range at first, then upper range if exists (else nothing), and then the property.
Then applyProperty is pretty simple, for each line we check if the property in the line matches the property passed as an argument, and if so it "turns on" the bit of that property for the unicode/unicodes (if a range was detected).

Then we applyProperty for each of the relevant options:

```
    applyProperty(emojiDataText, 'Emoji', FLAG_EMOJI);
    applyProperty(emojiDataText, 'Emoji_Presentation',
FLAG_EMOJI_PRESENTATION);
    applyProperty(eawText, 'W', FLAG_WIDE);
    applyProperty(eawText, 'F', FLAG_WIDE);
    applyProperty(eawText, 'A', FLAG_AMBIGUOUS);
```

And now the unicodeSpace array holds for each unicode if it's an emoji, a 2-width emoji, wide (W/F), or ambiguous.
Then compressToRanges gets a function that works on number and return a boolean, and returns the ranges of numbers that satisfy the function.
It starts a range once it hits a number that satisfy the function, and when it hits one that doesn't it closes the range and add it to the ranges list.
Now all left to do is define the lists we described above:

```
    // Wide emojis or wide chars
    const wideRanges = compressToRanges(val =>
        (val & FLAG_WIDE) !== 0 || (val & FLAG_EMOJI_PRESENTATION) !== 0
    );
    // Narrow emojis
    const narrowEmojiRanges = compressToRanges(val =>
        (val & FLAG_EMOJI) !== 0 && (val & FLAG_EMOJI_PRESENTATION) === 0
    );
    // ambigouos
    const ambiguousRanges = compressToRanges(val =>
        (val & FLAG_AMBIGUOUS) !== 0
    );
```

Now that we have the ranges, we can hardcode them to a file "ranges.ts" that will work the same as the ranges.ts on the previous version.
So we create a string that is the ts code that will be loaded to the file, which contains the hardcoded ranges and the functions we need (like isWide and such).
Now need to modify the width.ts logic as well:
1 - just use indices and match the standard of which comes last, instead of that vs15 always "wins".
So whenever a vs15 or vs16 shows up, it override the last one that was. And after processing the grapheme, the last one will be the one we remember.
2 - will be fixed while making correct lists at stage 4.
Indeed, we can tell now if isNarrowEmoji, and if there is not a vs16 later, it will be counted as 1.
3 - add an API option that allows to define behavior of such chars.
Bidi characters are now in isZeroWidthCodePoint. Control chars are now have width 0, except for tab which is 4 by default and the user can provide tab width as an option.
4 - in the next paragraph.
Done :)
7 - separate between invisible unicodes to visible that are like a mark on a letter, and if there is not letter assign in width of 1.

Instead of trying to hardcode the symbols that add to a char and should be width 1 only when they are alone, we just ignored them, and thus if they are alone they'll fall to the default and return 1 and if connected to a letter it will still return 1 and not add anything.

9 - in the process of solving 4 saving range of ambiguous unicodes as well.
Done. An option added.

So all problems of the last versions were solved :)
Note that fast calculation for ascii only strings is simple to add and can save time if assuming usage on ascii-only strings, but this is not the idea of the task the boring stuff so I passed :)